

**Universidade São Judas Tadeu**

# **Toad**

Manual da linguagem

**Joaquim Gomes de Moraes 82411680**

**Ryan Amorim dos Santos Silva 824146845**

**Fabio Junior Rodrigues Gomes 823119402**

## Operadores

| Operador                 | Exemplos                  | Função                                                                                               |
|--------------------------|---------------------------|------------------------------------------------------------------------------------------------------|
| Operadores relacionais   | >   >=   <   <=   ==   != | Símbolos utilizados para a comparação de dois valores ou expressões para resultar um valor booleano. |
| Operador de atribuição   | =                         | Símbolos utilizados para armazenar valores ou expressões na memória e atrelar a variáveis.           |
| Operadores aritméticos   | +   -   /   *   ^         | Símbolos utilizados para fazer operações matemáticas.                                                |
| Operadores entrada/saída | return   main             | Nomenclaturas utilizadas para inicializar uma entrada e saída de dados ou funções.                   |

## Estruturas

| Estruturas              | Exemplos         | Função                                                                 |
|-------------------------|------------------|------------------------------------------------------------------------|
| Estruturas de repetição | for   while   do | Estruturas utilizadas para a repetição de processos e funcionalidades. |
| Estruturas de decisão   | if   else        | Estruturas utilizadas para a tomada de decisão.                        |

## Bloco

| Símbolo | Função                                                                          |
|---------|---------------------------------------------------------------------------------|
| { }     | Utilizados para encapsular algoritmos e processar entradas e saídas de funções. |

## Agrupadores

| Símbolos | Função                                                                                  |
|----------|-----------------------------------------------------------------------------------------|
| () [] ;  | Utilizados para agrupar funções, organizar e gerenciar processos e correlacionar dados. |

## Palavras Reservadas

| Palavra reservada | Função                                                                                                     |
|-------------------|------------------------------------------------------------------------------------------------------------|
| for               | Controla fluxos de repetições de blocos e suas funções definidas e iterar coleções como por exemplo lista. |
| while             | Controla fluxo de repetições de blocos e suas funções definidas por verdadeiro e falso.                    |
| if                | Controla o fluxo de decisões de blocos, funcionando como um tomador de decisões.                           |

|        |                                                                                               |
|--------|-----------------------------------------------------------------------------------------------|
| do     | Dá início ao comando, exemplo faça até X.                                                     |
| else   | Controla também o fluxo de decisões, porém o resultado é contrário ao realizado na função if. |
| return | Utilizado para a saída de dados e finalização de um algoritmo.                                |
| printf | Exibir mensagens no console                                                                   |

## Comentário

| Símbolo | Função                                               |
|---------|------------------------------------------------------|
| //      | Utilizado para fazer comentários ao longo do código. |

## Números

| Tipos de Números | Expressões Regulares |
|------------------|----------------------|
| Números Inteiros | [0-9] +              |
| Números Reais    | [0-9] + \.[0-9] +    |

## Identificador

| Tipos de identificador | Padrão        |
|------------------------|---------------|
| Alfanumérico           | [a-zA-Z0-9_]* |

# Gramática

```
programa ::= listaInstrucao ;
listaInstrucao ::= instrucao+ ;
instrucao ::= declaracao ";" | atribuicao ";" | printf ";" | comando ;
declaracao ::= tipo identificador "=" expressao ;
atribuicao ::= identificador "=" expressao ;
tipo ::= "int" | "double" | "float" ;
printf ::= "printf" "(" printf_arg ")" ;
printfarg ::= expressao | identificador ;
bloco ::= "{" instrucao* "}";
comando ::= atribuicao ";" | comandoIfElse | comandoWhile | comandoFor |
comandoDo | comandoReturn | comentario ;
comandoIfElse ::= "if" "(" expressao ")" bloco ("else" bloco)? ;
comandoWhile ::= "while" "(" expressao ")" bloco ;
comandoFor ::= "for" "(" declaracao ";" condicao ";" expressao " | "i++" | "i - -" |
"atribuicao")" bloco ;
comandoDo ::= "do" bloco "while" "(" expressao ")" ";" ;
comandoReturn ::= "return" expressao? ";" ;
condicao ::= expressao operadorRelacional expressao ;
operadorRelacional ::= ">" | "<" | ">=" | "<=" | "==" | "!=" ;
expressao ::= condicao | soma ;
soma ::= termo (("+" | "-") termo)* ;
termo ::= fator (("*" | "/" ) fator) ;
fator ::= numeroReal | identificador | "(" expressao ")" ;
identificador ::= letra (letra | digito | "_")* ;
numeroReal ::= digito+ ( "." digito+ )? ;
letra ::= "a" | ... | "z" | "A" | ... | "Z" ;
digito ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
comentario ::= "/*" { qualquer_caractere } ;
```

## Exemplo Lexer

lexer.h

#pragma once

#include <string>

#include <vector>

enum class TokenType {

INT, FLOAT, DOUBLE, PRINTF,

IF, ELSE, WHILE, FOR, DO, RETURN,

IDENTIFICADOR, NUMERO, CADEIA,

MAIS, MENOS, ASTERISCO, BARRA, POTENCIA,

ATRIBUICAO, PONTO\_VIRGULA, ABRE\_PARENTESE,

FECHA\_PARENTESE,

ABRE\_CHAVES, FECHA\_CHAVES,

MAIOR, MENOR, MAIOR\_IGUAL, MENOR\_IGUAL, IGUAL\_IGUAL,  
DIFERENTE,

FIM

};

struct Token {

TokenType type;

std::string value;

int line;

};

class Lexer {

std::string src;

size\_t pos = 0;

int line = 1;

public:

explicit Lexer(std::string source) : src(std::move(source)) {}

Token nextToken();

private:

void skipWhitespace();

void skipComment();

std::string readIdentifier();

std::string readNumber();

std::string readString();

bool isKeyword(const std::string& s);

};

lexer.cpp

```
#include "lexer.h"
```

```
#include <cctype>
```

```
#include <stdexcept>
```

```
bool Lexer::isKeyword(const std::string& s) {
```

```
    if (s == "int") return true;
```

```
    if (s == "float") return true;
```

```
    if (s == "double") return true;
```

```
    if (s == "printf") return true;
```

```
    if (s == "if") return true;
```

```
    if (s == "else") return true;
```

```
    if (s == "while") return true;
```

```
    if (s == "for") return true;
```

```
    if (s == "do") return true;
```

```
    if (s == "return") return true;
```

```
    return false;
```

```
}
```

```
Token Lexer::nextToken() {
```

```
    while (true) {
```

```
        skipWhitespace();
```

```
        if (pos >= src.size()) return {TokenType::FIM, "", line};
```

```
        if (src[pos] == '/' && pos + 1 < src.size() && src[pos+1] == '/') {
```

```
            skipComment();
```

```
            continue;
```

```
        }
```

```
        break;
```

```
    }
```

```
    if (pos >= src.size()) return {TokenType::FIM, "", line};
```

```
    char c = src[pos];
```

```
    Token tok; tok.line = line;
```

```
    if (std::isalpha(static_cast<unsigned char>(c)) || c == '_') {
```

```
        tok.value = readIdentifier();
```

```
        if (tok.value == "int") tok.type = TokenType::INT;
```

```
        else if (tok.value == "float") tok.type = TokenType::FLOAT;
```

```
        else if (tok.value == "double") tok.type = TokenType::DOUBLE;
```

```
        else if (tok.value == "printf") tok.type = TokenType::PRINTF;
```

```
        else if (tok.value == "if") tok.type = TokenType::IF;
```

```
        else if (tok.value == "else") tok.type = TokenType::ELSE;
```

```
        else if (tok.value == "while") tok.type = TokenType::WHILE;
```

```

else if (tok.value == "for") tok.type = TokenType::FOR;
else if (tok.value == "do") tok.type = TokenType::DO;
else if (tok.value == "return") tok.type = TokenType::RETURN;
else tok.type = TokenType::IDENTIFICADOR;
} else if (std::isdigit(static_cast<unsigned char>(c))) {
    tok.value = readNumber();
    tok.type = TokenType::NUMERO;
} else if (c == '"') {
    tok.value = readString();
    tok.type = TokenType::CADEIA;
} else {
    switch (c) {
        case '+': tok.type = TokenType::MAIS; tok.value = "+"; pos++; break;
        case '-': tok.type = TokenType::MENOS; tok.value = "-"; pos++; break;
        case '*': tok.type = TokenType::ASTERISCO; tok.value = "*"; pos++; break;
        case '/': tok.type = TokenType::BARRA; tok.value = "/"; pos++; break;
        case '^': tok.type = TokenType::POTENCIA; tok.value = "^"; pos++; break;
        case ';': tok.type = TokenType::PONTO_VIRGULA; tok.value = ";"; pos++;
break;
        case '(': tok.type = TokenType::ABRE_PARENTESE; tok.value = "(";
pos++; break;
        case ')': tok.type = TokenType::FECHA_PARENTESE; tok.value = ")";
pos++; break;
        case '{': tok.type = TokenType::ABRE_CHAVES; tok.value = "{"; pos++;
break;
        case '}': tok.type = TokenType::FECHA_CHAVES; tok.value = "}"; pos++;
break;
        case '>':
            pos++;
            if (pos < src.size() && src[pos] == '=') {
                tok.type = TokenType::MAIOR_IGUAL;
                tok.value = ">=";
                pos++;
            } else {
                tok.type = TokenType::MAIOR;
                tok.value = ">";
            }
            break;
        case '<':
            pos++;
            if (pos < src.size() && src[pos] == '=') {
                tok.type = TokenType::MENOR_IGUAL;
                tok.value = "<=";
                pos++;
            } else {

```



```

        tok.type = TokenType::MENOR;
        tok.value = "<";
    }
    break;
case '=':
    pos++;
    if (pos < src.size() && src[pos] == '=') {
        tok.type = TokenType::IGUAL_IGUAL;
        tok.value = "==";
        pos++;
    } else {
        tok.type = TokenType::ATRIBUICAO;
        tok.value = "=";
    }
    break;
case '!':
    pos++;
    if (pos < src.size() && src[pos] == '=') {
        tok.type = TokenType::DIFERENTE;
        tok.value = "!=";
        pos++;
    } else {
        throw std::runtime_error("Caractere inválido na linha " +
std::to_string(line));
    }
    break;
default:
    throw std::runtime_error("Caractere inválido na linha " +
std::to_string(line));
    }
}
return tok;
}

void Lexer::skipWhitespace() {
    while (pos < src.size() && std::isspace(static_cast<unsigned char>(src[pos]))) {
        if (src[pos] == '\n') line++;
        pos++;
    }
}

void Lexer::skipComment() {
    pos += 2; // pula '/'
    while (pos < src.size() && src[pos] != '\n') pos++;
}

```

```
std::string Lexer::readIdentifier() {
    size_t start = pos;
    while (pos < src.size() && (std::isalnum(static_cast<unsigned char>(src[pos])) ||
src[pos] == '_')) pos++;
    return src.substr(start, pos - start);
}

std::string Lexer::readNumber() {
    size_t start = pos;
    bool hasDot = false;
    while (pos < src.size() && (std::isdigit(static_cast<unsigned char>(src[pos])) ||
(!hasDot && src[pos] == '.'))) {
        if (src[pos] == '.') hasDot = true;
        pos++;
    }
    return src.substr(start, pos - start);
}

std::string Lexer::readString() {
    pos++; // pula a aspa de abertura
    size_t start = pos;
    while (pos < src.size() && src[pos] != '"') pos++;
    std::string s = src.substr(start, pos - start);
    pos++; // pula a aspa de fechamento
    return s;
}
```